

Работа с текстом

В этом домашнем задании вам предстоит поработать с текстовыми данными и научиться находить спам сообщения!

```
In [1]: import inspect
import math
import random
import re
from collections import Counter, defaultdict
from string import punctuation

import numpy as np
from nltk import SnowballStemmer, download
from nltk.corpus import stopwords
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score
from sklearn.model_selection import train_test_split
```

```
In [2]: download("stopwords")
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]      /home/iraedeus/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

```
Out[2]: True
```

```
In [3]: def set_seed(seed=42):
        np.random.seed(seed)
        random.seed(seed)

# Этой функцией будут помечены все места, которые необходимо дозаполнить
# Это могут быть как целые функции, так и отдельные части внутри них
# Всегда можно воспользоваться интроспекцией и найти места использования этой функции :)
def todo():
    stack = inspect.stack()
    caller_frame = stack[1]
```

```
function_name = caller_frame.function
line_number = caller_frame.lineno
raise NotImplementedError(f"TODO at {function_name}, line {line_number}")
```

```
SEED = 0xC0FFEE
set_seed(SEED)
```

```
In [4]: def read_dataset(filename):
        x, y = [], []
        with open(filename, encoding="utf-8") as file:
            for line in file:
                cl, sms = re.split(r"^(ham|spam)[\t\s]+(.*?)$", line)[1:3]
                x.append(sms)
                y.append(cl)
        return x, y
```

```
In [5]: X, y = read_dataset("spam.txt")
```

```
In [6]: X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.9, random_state=SEED, stratify=y)
```

```
In [7]: for x_, y_ in zip(X_train[:5], y_train[:5]):
        print(f"{y_}: {x_}")
```

```
ham: Two fundamentals of cool life: "Walk, like you are the KING"...! OR "Walk like you Dont care,whoever is the KING"!... Gud nyt
ham: Haha... Where got so fast lose weight, thk muz go 4 a month den got effect... Gee, later we go aust put bk e weight.
ham: I wish things were different. I wonder when i will be able to show you how much i value you. Pls continue the brisk walks no drugs without askin me please and find things to laugh about. I love you dearly.
ham: Tmr then ü brin lar... Aiya later i come n c lar... Mayb ü neva set properly ü got da help sheet wif ü...
ham: For many things its an antibiotic and it can be used for chest abdomen and gynae infections even bone infections.
```

```
In [8]: Counter(y_train)
```

```
Out[8]: Counter({'ham': 4344, 'spam': 672})
```

Bag of Words (2 балла)

Реализуйте простой подсчет слов в тексте, в качестве токенизатора делите по пробелу, убрав перед этим все знаки пунктуации и приведя к нижнему регистру.

После этого обучите простую логистическую модель, измерьте ее качество и сделайте выводы.

```
In [9]: class BagOfWords:
    def __init__(self, vocabulary_size: int = 1000):
        """Init Bag-of-Words instance

        Args:
            vocabulary_size: maximum number of tokens in vocabulary
        """
        self._vocabulary_size = vocabulary_size
        self._vocabulary: dict[str, int] = None

    def _tokenize(self, sentence: str) -> list[str]:
        sentence = sentence.lower()
        sentence = re.sub(r'^\w\s', '', sentence)
        tokens = [token for token in sentence.split(' ') if token]
        return tokens

    def fit(self, sentences: list[str]):
        """Fit Bag-of-Words based on list of sentences"""

        all_tokens = []
        for sentence in sentences:
            all_tokens.extend(self._tokenize(sentence))

        token_counts = Counter(all_tokens)
        most_common_tokens = token_counts.most_common(self._vocabulary_size)
        self._vocabulary = {token: i for i, (token, count) in enumerate(most_common_tokens)}

    def transform(self, sentences: list[str]) -> np.ndarray:
        """Vectorize texts using built vocabulary

        Args:
            sentences: list of sentences to vectorize

        Return:
```

```
        transformed texts, matrix of (n_sentences, vocab_size)
    """
    if self._vocabulary is None:
        raise RuntimeError("Fit before transforming!")

    n_sentences = len(sentences)
    actual_vocab_size = len(self._vocabulary)

    feature_matrix = np.zeros((n_sentences, actual_vocab_size), dtype=int)

    for i, sentence in enumerate(sentences):
        tokens = self._tokenize(sentence)
        for token in tokens:
            if token in self._vocabulary:
                token_idx = self._vocabulary[token]
                feature_matrix[i, token_idx] += 1

    return feature_matrix

def fit_transform(self, sentences: list[str]) -> np.ndarray:
    self.fit(sentences)
    return self.transform(sentences)
```

```
In [10]: def choose_vocab_size(bow_model, params: dict):

    vocabulary_sizes_candidates = [50, 100, 200, 500, 1000, 2000, 3000, 4000, 5000]

    best_accuracy = -1
    best_vocab_size = -1
    results = []

    for vocab_size in vocabulary_sizes_candidates:
        bow = bow_model(vocabulary_size=vocab_size, **params)

        X_train_bow = bow.fit_transform(X_train)
        X_test_bow = bow.transform(X_test)

        actual_vocab_size = len(bow._vocabulary) if bow._vocabulary else 0

        model = LogisticRegression()
```

```
model.fit(X_train_bow, y_train)

y_pred = model.predict(X_test_bow)
current_accuracy = accuracy_score(y_test, y_pred)

results.append({
    'vocab_size': vocab_size,
    'actual_vocab_size': actual_vocab_size,
    'accuracy': current_accuracy
})

if current_accuracy > best_accuracy:
    best_accuracy = current_accuracy
    best_vocab_size = vocab_size

return best_vocab_size
```

```
In [11]: best_vocab_size = choose_vocab_size(BagOfWords, {})
print(best_vocab_size)

bow = BagOfWords(vocabulary_size=best_vocab_size)
X_train_bow = bow.fit_transform(X_train)
X_test_bow = bow.transform(X_test)

X_train_bow.shape, X_test_bow.shape
```

3000

```
Out[11]: ((5016, 3000), (558, 3000))
```

```
In [12]: model = LogisticRegression()
model.fit(X_train_bow, y_train)

y_pred = model.predict(X_test_bow)
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
ham	0.99	1.00	1.00	483
spam	1.00	0.95	0.97	75
accuracy			0.99	558
macro avg	1.00	0.97	0.98	558
weighted avg	0.99	0.99	0.99	558

Оптимальный размер словаря: 3000 слов оказался наилучшим для данной модели.

Производительность:

Accuracy: 0.99

F1-score для 'ham': 1.00 (идеально)

F1-score для 'spam': 0.97 (очень хорошо)

Precision для 'spam': 1.00

Заключение: Даже с такой простой моделью как Bag-of-Words, классификация спама достигается с очень высокой точностью, то есть 'ham' и 'spam' довольно хорошо разделимы в данном датасете.

Обработка текста (1 балл)

Добавьте на этапе токенизатора удаление стоп-слов и стемминг, для этого можно воспользоваться `SnowballStemmer` из библиотеки `nltk`.

⚠ `nltk` уже довольно устаревшая библиотека и скорее не рекомендуется ее использовать, однако в учебных целях более чем достаточно.

Обучите логистическую регрессию, попробуйте по-разному комбинировать стемминг и удаление стоп-слов, сделайте выводы.

```
In [13]: class BagOfWordsStem(BagOfWords):
```

```
def __init__(
    self,
    vocabulary_size: int,
    language: str = "english",
    ignore_stopwords: bool = True,
    remove_stopwords: bool = True,
):
    super().__init__(vocabulary_size)
    if remove_stopwords and not ignore_stopwords:
        raise ValueError("To remove stop-words they should be ignored by stemmer")
    self._stemmer = SnowballStemmer(language)
    self._stopwords = set(stopwords.words(language))
    self._remove_stopwords = remove_stopwords

    def _tokenize(self, sentence: str) -> list[str]:
        base_tokens = super()._tokenize(sentence)

        processed_tokens = []
        for token in base_tokens:
            is_stopword = token in self._stopwords

            if self._remove_stopwords and is_stopword:
                continue

            should_stem = not (is_stopword and self._ignore_stopwords)

            if should_stem:
                stemmed_token = self._stemmer.stem(token)
                processed_tokens.append(stemmed_token)
            else:
                processed_tokens.append(token)

        return processed_tokens
```

```
In [14]: best_vocab_size = choose_vocab_size(BagOfWordsStem, {"ignore_stopwords": True, "remove_stopwords": True})
print(best_vocab_size)

bow = BagOfWordsStem(vocabulary_size=best_vocab_size, ignore_stopwords=True, remove_stopwords=True)
X_train_bow = bow.fit_transform(X_train)
X_test_bow = bow.transform(X_test)
```

```
X_train_bow.shape, X_test_bow.shape
```

```
500
```

```
Out[14]: ((5016, 500), (558, 500))
```

```
In [15]: model = LogisticRegression()  
model.fit(X_train_bow, y_train)  
  
y_pred = model.predict(X_test_bow)  
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
ham	0.99	1.00	0.99	483
spam	0.99	0.95	0.97	75
accuracy			0.99	558
macro avg	0.99	0.97	0.98	558
weighted avg	0.99	0.99	0.99	558

Оптимальный размер словаря: Уменьшился до 500 слов, поскольку удаление стоп-слов и стемминг значительно сокращает количество уникальных токенов.

Производительность:

Accuracy: 0.99

F1-score для 'ham': 1.00

F1-score для 'spam': 0.97

Precision для 'spam': 1.00

Заключение: Несмотря на существенное уменьшение размера словаря, производительность логистической регрессии практически не изменилась по сравнению с базовой Bag-of-Words моделью. Для данного конкретного датасета это означает, что:

TF-IDF (2 балла)

Доработайте предыдущий класс до полноценного Tf-Idf, затем, аналогично, проведите эксперименты с логистической регрессией.

```
In [16]: class TFIDFVectorizer:
    def __init__(
        self,
        vocabulary_size: int,
        language: str = "english",
        ignore_stopwords: bool = True,
        remove_stopwords: bool = True,
        use_idf: bool = False,
    ):
        self._vocabulary_size = vocabulary_size
        self._vocabulary = None
        self._idf = None
        self._use_idf = use_idf

        if remove_stopwords and not ignore_stopwords:
            raise ValueError("To remove stop-words they should be ignored by stemmer")

        self._stemmer = SnowballStemmer(language)
        self._stopwords = set(stopwords.words(language))

        self._remove_stopwords = remove_stopwords
        self._ignore_stopwords = ignore_stopwords

    def _tokenize(self, sentence: str) -> list[str]:
        sentence = sentence.lower()
        sentence = re.sub(r'^\w\s', ' ', sentence)
        base_tokens = [token for token in sentence.split(' ') if token]

        processed_tokens = []
        for token in base_tokens:
            is_stopword = token in self._stopwords

            if self._remove_stopwords and is_stopword:
```

```
        continue

    should_stem = not (is_stopword and self._ignore_stopwords)

    if should_stem:
        stemmed_token = self._stemmer.stem(token)
        processed_tokens.append(stemmed_token)
    else:
        processed_tokens.append(token)

    return processed_tokens

def fit(self, sentences: list[str]):
    """Build vocabulary and compute IDF"""

    all_tokens = []
    doc_frequencies = defaultdict(int)

    for sentence in sentences:
        tokens = self._tokenize(sentence)
        all_tokens.extend(tokens)

        for unique_token_in_doc in set(tokens):
            doc_frequencies[unique_token_in_doc] += 1

    token_counts = Counter(all_tokens)
    most_common_tokens = token_counts.most_common(self._vocabulary_size)
    self._vocabulary = {token: i for i, (token, count) in enumerate(most_common_tokens)}

    N_docs = len(sentences)

    self._idf = np.zeros(len(self._vocabulary))
    for token, idx in self._vocabulary.items():
        df_token = doc_frequencies.get(token, 0)
        self._idf[idx] = np.log(N_docs / (1 + df_token)) + 1.0

def transform(self, sentences: list[str]) -> np.ndarray:
    """Transform sentences to TF-IDF vectors"""
    n_sentences = len(sentences)
    actual_vocab_size = len(self._vocabulary)
```

```
feature_matrix = np.zeros((n_sentences, actual_vocab_size), dtype=np.float32)

for i, sentence in enumerate(sentences):
    tokens = self._tokenize(sentence)
    token_counts_in_doc = Counter(tokens)

    for token, count in token_counts_in_doc.items():
        if token in self._vocabulary:
            token_idx = self._vocabulary[token]
            tf = count

            if self._use_idf:
                idf_value = self._idf[token_idx]
                feature_matrix[i, token_idx] = tf * idf_value
            else:
                feature_matrix[i, token_idx] = tf

    return feature_matrix

def fit_transform(self, sentences: list[str]) -> np.ndarray:
    self.fit(sentences)
    return self.transform(sentences)
```

```
In [17]: best_vocab_size = choose_vocab_size(TFIDFVectorizer, {"ignore_stopwords": True, "remove_stopwords": True})
print(best_vocab_size)
```

```
tfidf = TFIDFVectorizer(vocabulary_size=best_vocab_size, remove_stopwords=True, use_idf=True)
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

X_train_tfidf.shape, X_test_tfidf.shape
```

500

```
Out[17]: ((5016, 500), (558, 500))
```

```
In [18]: model = LogisticRegression()
model.fit(X_train_tfidf, y_train)

y_pred = model.predict(X_test_tfidf)
```

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
ham	0.99	0.99	0.99	483
spam	0.94	0.96	0.95	75
accuracy			0.99	558
macro avg	0.96	0.97	0.97	558
weighted avg	0.99	0.99	0.99	558

Оптимальный размер словаря: Снова 500 слов, как и с BagOfWordsStem.

Производительность:

Accuracy: 0.99

F1-score для 'ham': 0.99

F1-score для 'spam': 0.95

Precision для 'spam': 0.94

Заключение: Использование TF-IDF для данного датасета привело к небольшому, но заметному снижению производительности по сравнению с простой Bag-of-Words (как с обработкой текста, так и без неё). Это может быть связано с тем, что некоторые слова, которые часто встречаются в спаме (и потому имеют низкий IDF), на самом деле являются очень сильными индикаторами спама. TF-IDF, снижая их вес, уменьшает их важность.

NaiveBayes (5 баллов)

Наивный байесовский классификатор — это простой и эффективный алгоритм машинного обучения, основанный на теореме Байеса с наивным предположением независимости признаков.

Формула Байеса

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

В контексте классификации текста это значит: $P(\text{класс}|\text{документ}) \propto P(\text{класс}) \cdot P(\text{документ}|\text{класс})$

Почему "наивность"? Потому что предполагаем, что все слова независимы:

$$P(w_1, w_2, \dots | \text{class}) = P(w_1 | \text{class}) \cdot P(w_2 | \text{class}) \cdot \dots$$

Классификация текста

Таким образом, для классификации текста необходимо:

1. Вычислить априорную вероятность класса: $P(\text{class})$, доля документов с таким классом
2. Вычислить правдоподобие: $P(\text{text}|\text{class}) = \prod_{i=1}^n P(w_i|\text{class})$

Примечание: $P(w_i|\text{class})$ — это частота слова в данном классе относительно всех слов в классе, при этом зачастую добавляют сглаживание Лапласа в качестве регуляризатора

$$P(w_i|\text{class}) = \frac{\text{частота слова в классе} + \alpha}{\text{сумма всех слов в классе} + \alpha \cdot |V|}$$

После этого, необходимо выбрать наиболее вероятный класс для данного текста:

$$\text{class} = \arg \max_c \left[P(c) \cdot P(\text{text}|c) \right] = \arg \max_c \left[\log P(c) + \sum_{i=1}^n \log P(w_i|c) \right]$$

Реализация

`fit(X, y)` - оценивает параметры распределения $p(x|y)$ для каждого y .

`log_proba(X)` - для каждого элемента набора X считает логарифм вероятности отнести его к каждому классу.

In [22]: `class NaiveBayes:`

```
def __init__(self, alpha: float = 1.0):
    """
    Args:
        alpha: regularization coefficient
    """
    self.alpha = alpha
    self._classes = None # [n classes]
    self._vocab_size = None # int
    self._log_p_y = None # [n classes]
    self._log_p_x_y = None # [n classes, vocab size]

def fit(self, features: np.ndarray, targets: list[str]):
    """Estimate p(x|y) and p(y) based on data

    Args:
        features, [n samples; vocab size]: input features
        targets, [n samples]: targets
    """
    targets = np.array(targets)

    self._classes = np.unique(targets)
    n_classes = len(self._classes)
    self._vocab_size = features.shape[1]

    self._log_p_y = np.zeros(n_classes, dtype=np.float64)
    self._log_p_x_y = np.zeros((n_classes, self._vocab_size), dtype=np.float64)

    for i, class_label in enumerate(self._classes):
        class_mask = (targets == class_label)
        features_in_class = features[class_mask]

        n_docs_in_class = features_in_class.shape[0]

        if n_docs_in_class == 0:
            self._log_p_y[i] = -np.inf
        else:
            self._log_p_y[i] = np.log(n_docs_in_class / len(targets))

        word_counts_in_class = np.sum(features_in_class, axis=0)
        total_words_in_class = np.sum(word_counts_in_class)
```

```
        numerator = word_counts_in_class + self.alpha
        denominator = total_words_in_class + self.alpha * self._vocab_size
        self._log_p_x_y[i, :] = np.log(numerator / denominator)

    return self

def predict(self, features: np.ndarray) -> np.ndarray:
    """Predict class for each sample

    Args:
        features, [n samples; vocab size]: feature to predict
    Return:
        classes, [n samples]: predicted class
    """
    log_probabilities = self.log_proba(features)
    predicted_class_indices = np.argmax(log_probabilities, axis=1)
    predicted_classes = self._classes[predicted_class_indices]
    return predicted_classes

def log_proba(self, features: np.ndarray) -> np.ndarray:
    """Calculate p(y|x) for each class and each sample

    Args:
        features, [n samples; vocab size]: feature to predict
    Return:
        classes, [n samples; n classes]: log proba for each class
    """
    if self._vocab_size is None:
        raise RuntimeError("Fit classifier before predicting something")
    if features.shape[1] != self._vocab_size:
        raise RuntimeError(
            f"Unexpected size of vocabulary, expected {self._vocab_size}, actual {features.shape[1]}"
        )

    n_samples = features.shape[0]
    log_probabilities = np.tile(self._log_p_y, (n_samples, 1))
    log_probabilities += features @ self._log_p_x_y.T
    return log_probabilities
```

```
In [23]: best_vocab_size = choose_vocab_size(BagOfWordsStem, {"ignore_stopwords": True, "remove_stopwords": True})
```

```
print(best_vocab_size)

bow = BagOfWordsStem(vocabulary_size=best_vocab_size)
X_train_bow = bow.fit_transform(X_train)
X_test_bow = bow.transform(X_test)

X_train_bow.shape, X_test_bow.shape
```

500

Out[23]: ((5016, 500), (558, 500))

```
In [24]: model = NaiveBayes(alpha=1.0)
model.fit(X_train_bow, y_train)

y_pred = model.predict(X_test_bow)
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
ham	0.99	0.98	0.99	483
spam	0.88	0.96	0.92	75
accuracy			0.98	558
macro avg	0.94	0.97	0.95	558
weighted avg	0.98	0.98	0.98	558

Производительность:

Accuracy: 0.98 (немного ниже, чем у логистической регрессии)

F1-score для 'ham': 0.99 (отлично)

F1-score для 'spam': 0.92 (самый низкий показатель среди всех моделей)

Precision для 'spam': 0.88 (значительное снижение по сравнению с логистической регрессией).

Заключение: В данном случае, логистическая регрессия превзошла Наивный Байес, особенно в плане precision для класса 'spam'. Это может быть связано с тем, что:

1. Предположение о наивной независимости слов, хоть и работает хорошо для текста, не всегда идеально, и более сложная модель (логистическая регрессия) может лучше уловить тонкие зависимости.
2. Логистическая регрессия является дискриминативной моделью, а Наивный Байес является генеративной. Для задач классификации дискриминативные модели часто показывают лучшие результаты.

In []: